Introduction to Artificial Intelligence

Project in Search Algorithms

Submitters:

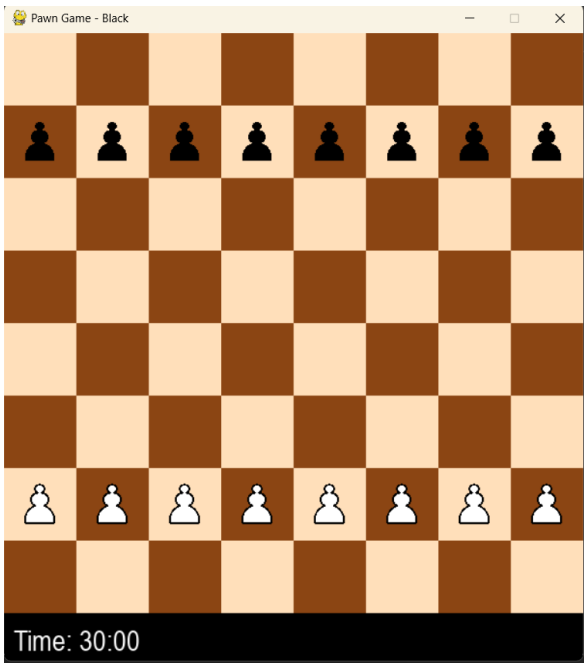
Shatha Maree ; 211434196

Lian Natour ; 207300443

1. Questions about Representation and Data Structures:

a. What representation did you choose for the game board?

The game board is represented as an 8x8 2D array (boardArray) in the ChessBoard class. Each cell in the array can either be empty ("--"), contain a white pawn ("wp"), or contain a black pawn ("bp"), which is also shown as a GUI board.



b. What data structures does the agent rely on?

The agent uses the following data structures:

- **2D Array**: To represent the board (boardArray).
- **Dictionary**: To store the transposition table (transposition_table) for caching previously evaluated board states.
- **List**: To store possible moves and sort them based on heuristic values.
- **Dictionary (weights)**:Used to assign weights to different features of the game, such as pawn advancement, passed pawns, and captures .

c. How does the algorithm generate its moves?

The algorithm generates moves by iterating over all pawns on the board and checking all possible moves for each pawn based on the game rules. It uses the getPossibleMoves method to generate moves.

d. How does the algorithm identify terminal states?

The algorithm identifies terminal states by checking if a pawn has reached the last row, if all opponent pawns have been captured, or if there are no legal moves left. This is done in the isGameOver method.

e. How does the algorithm manage the time allocated for the entire game?

The algorithm uses iterative deepening with a time limit. It starts with a depth of 1 and increases the depth until the time limit is about to be exceeded. This is managed in the getBestMove method.

```
# Iterative deepening: Start with depth 1 and increase until time runs out
for depth in range(1, max_depth + 1):
    elapsed_time = time.time() - start_time
    if elapsed_time > time_limit:
        break  # Stop searching if time limit is exceeded
```

f. Does the algorithm think while the opponent is thinking?

No, the algorithm does not think while the opponent is thinking. It only performs calculations during its own turn using iterative deepening to search for the best move within the allocated time limit.

2. **Questions about the Evaluation Function:**

a. Describe the static evaluation function you built.

The static evaluation function (heuristic) evaluates the board state based on several factors, including material advantage, pawn advancement, and hanging pawns. It assigns weights to each factor based on their importance. The best state (of a board) that is chosen, is obviously the state with the higher evaluation value.

b. Which features of the game does it include?

The evaluation function includes features such as:

- Material Advantage: Difference in the number of pawns between both players.
- Pawn Advancement: Bonus for pawns closer to promotion.
- Passed Pawns: Bonus for pawns that are not blocked by opponent pawns – by blocked we mean that the three columns (if existed) around the pawn, starting from his current row to his finish line don't include any of the opponent's pieces that might block him.
- Captures: Bonus for capturing opponent pawns.
- Hanging pawns: penalty for hanging pawns

```
weights = {
    "win_score": 100000,  # Winning the game
    "diff_pawn": 20,  # Difference in number of pawns
    "promote_pawn": 500,  # # Huge bonus for promoting pawns
    "advanced_row": 40,  # bonus for moving pawns forward
    "passed_pawn": 250,  # bigger bonus for having a passed pawn
    "white_advancement_pawn": 40,  # Penalty for White pawns advanced pawns (as a black player)
    "black_advancement_pawn": 40,  # Penalty for White advanced pawns (as a white player)
    "white_passed_pawn": 180,  # bigger penalty for letting White have a passed pawn
    "black_passed_pawn": 180,  # bigger penalty for letting black have a passed pawn
    "capture_opponent": 50  # Gains for capturing opponent's pawns
}
```

c. How does it extract these features from the current board state?

The function iterates over the board to count the number of pawns and checks their positions relative to the center and the last row. It also checks for passed pawns and captures. Meaning that for each pawn, we check if he is passed, if he can capture, and so on. Based on these results, the score of the board is calculated.

d. How does it weight these features?

The evaluation function in the agent uses a weighted sum of various features to assign a score to a given board state. Each feature is assigned a specific weight, which determines its relative importance in the overall evaluation. These weights are defined in a dictionary called weights within the Agent class.

Weights of the positions have been decided by trial and error: our agent played against Shay's agent, also against other students' agents, and after each game we were able to assign more accurate weights to the positions, and that's how we managed to get the best weights for the heuristic function.

Here's how the features are weighed:

<u>Win Score (win_score):</u>

This is the most heavily weighted feature. If the agent wins , it adds 100,000 to the score. This ensures that winning is always prioritized over other considerations.

<u>Pawn Difference (diff_pawn):</u>

The difference in the number of pawns between the agent and the opponent is multiplied by 20. For example, if the agent has 2 more pawns than the opponent, this adds 40 to the score.

<u>Promotion Bonus (promote_pawn):</u>

If a pawn is close to promotion (one move away from reaching the last row), it adds 500 to the score. This encourages the agent to prioritize promoting pawns.

<u>Advanced Pawns (advanced_row):</u>

Each pawn that has advanced further up the board adds 40 multiplied by its row distance from the starting position. For example, a White pawn on row 3 (5 rows from the starting position) adds 200 to the score.

<u>Passed Pawns (passed_pawn):</u>

If a pawn is a passed pawn (no opposing pawns blocking its path to promotion), it adds 250 to the score. This encourages the agent to create and protect passed pawns.

<u>Opponent's Advanced Pawns (white_advancement_pawn, black_advancement_pawn):</u>

If the opponent has pawns that are advanced, the agent subtracts 40 multiplied by the row distance of those pawns. This discourages the agent from allowing the opponent to advance their pawns.

<u>Opponent's Passed Pawns (white_passed_pawn, black_passed_pawn):</u>

If the opponent has a passed pawn, the agent subtracts 180 from the score. This discourages the agent from allowing the opponent to create passed pawns.

<u>Capture Bonus (capture_opponent):</u>

If the agent captures an opponent's pawn, it adds 50 to the score. This encourages the agent to capture opponent pawns when possible.

e. What is the range of the evaluation function?

The evaluation function returns values in a wide range, depending on the board state. For example, a win (win_score) is valued at 100000, while smaller features like pawn advancement are valued in the tens or hundreds.

f. What is the value of the function in a terminal state?

In a terminal state where the player wins, the function returns 100000. If the player loses, it returns -100000.

g. How did you test the accuracy of the function?

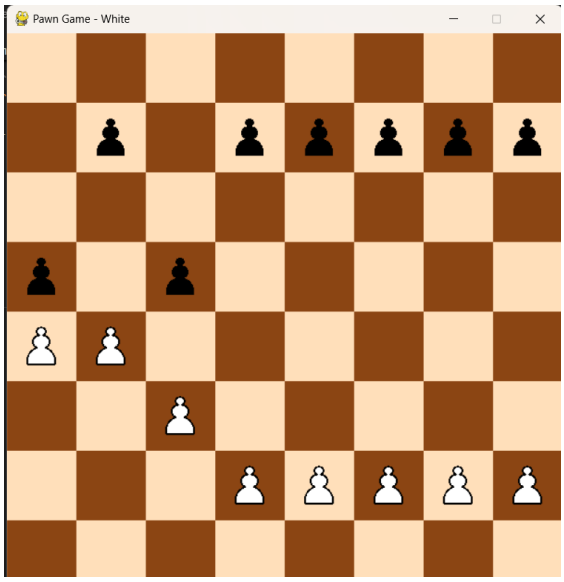
The accuracy of the evaluation function was tested by comparing its evaluations with the outcomes of actual games. Adjustments were made to the weights based on these tests.

h. Demonstrate the evaluation of the function for several key game states that you used to test it.

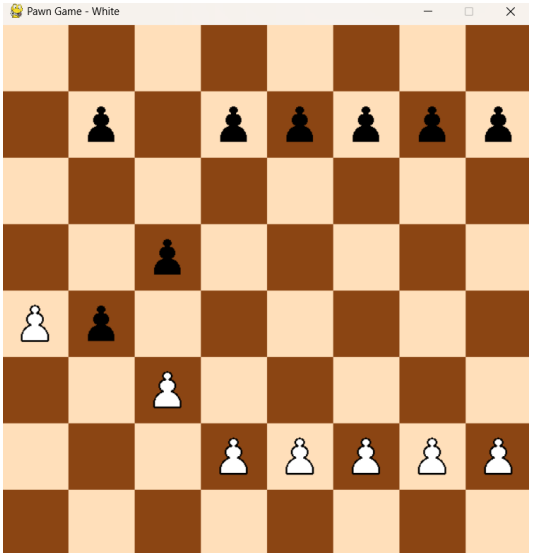
(For example, in a state where a pawn is close to promotion, the function returns a high value. In a state where all pawns are blocked, it returns a low value.)

In order to demonstrate how our heuristic function evaluates, let's illustrate a game that starts with full basic setup, where a black agent plays against a human. In the following state:

As shown, the best move that our agent could choose is to capture the white pawn. We printed the values that were calculated for each move in the heuristic function, and as seen in the next image, we got maximum evaluation for the moves of (3,0) to (4,1) and from (3,2) to (4,1), which are both for capturing the white pawn on (4,1) by the two black pawns. So indeed, our agent chooses to move from (3,0) to (4,1), as shown in the last image.

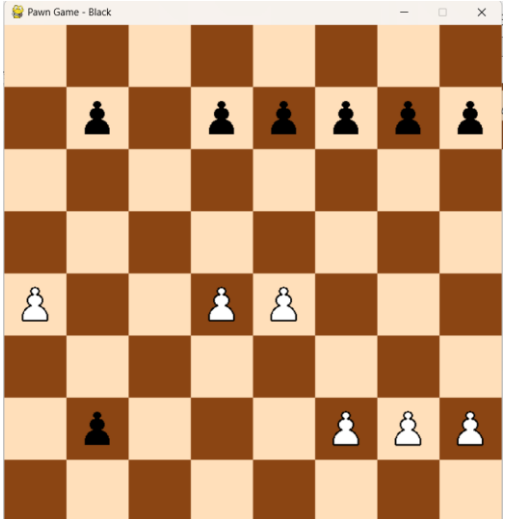


```
Move: (3, 0, 4, 1), Evaluation: 120
Move: (3, 2, 4, 1), Evaluation: 120
Move: (1, 3, 3, 3), Evaluation: 0
Move: (1, 4, 3, 4), Evaluation: 0
Move: (1, 5, 3, 5), Evaluation: 0
Move: (1, 6, 3, 6), Evaluation: 0
Move: (1, 7, 3, 7), Evaluation: 0
Move: (1, 1, 3, 1), Evaluation: -20
Move: (3, 2, 4, 2), Evaluation: -20
Move: (1, 1, 2, 1), Evaluation: -40
Move: (1, 3, 2, 3), Evaluation: -40
Move: (1, 4, 2, 4), Evaluation: -40
Move: (1, 5, 2, 5), Evaluation: -40
Move: (1, 6, 2, 6), Evaluation: -40
Move: (1, 7, 2, 7), Evaluation: -40
```



Another example is when the black pawn reaches one row before the last one, and as shown the move for (5,1) to (6,1) indeed had the highest evaluation because he is a passed, promoted pawn.

```
Move: (5, 1, 6, 1), Evaluation: 984.2857142857142
Move: (1, 6, 3, 6), Evaluation: 488.57142857142867
Move: (1, 7, 3, 7), Evaluation: 488.57142857142867
Move: (1, 1, 3, 1), Evaluation: 468.57142857142867
Move: (1, 3, 3, 3), Evaluation: 468.57142857142867
Move: (1, 4, 3, 4), Evaluation: 468.57142857142867
Move: (1, 5, 3, 5), Evaluation: 468.57142857142867
Move: (1, 1, 2, 1), Evaluation: 448.57142857142867
Move: (1, 3, 2, 3), Evaluation: 448.57142857142867
Move: (1, 4, 2, 4), Evaluation: 448.57142857142867
Move: (1, 5, 2, 5), Evaluation: 448.57142857142867
Move: (1, 6, 2, 6), Evaluation: 448.57142857142867
Move: (1, 7, 2, 7), Evaluation: 448.57142857142867
```



i. Do you use it only at leaf nodes or also for pruning and/or internal nodes?

The evaluation function is used at leaf nodes and also for alpha-beta pruning to cut off branches that are unlikely to be beneficial.

3. **Questions about the Search Algorithm:**

a. Describe the search algorithm you chose.

The algorithm uses Minimax with Alpha-Beta Pruning to search for the best move. It also uses iterative deepening to manage the time limit.

b. Did you use search heuristics?

Yes, the algorithm uses heuristics to prioritize moves that advance pawns toward the last row and to evaluate board states.

```
moves = self.getPossibleMoves(player)
moves.sort(key=lambda move: self.heuristic(self.simulateMove(board, move), player), reverse=True)
```

c. Did you use a transposition table?

Yes, the algorithm uses a transposition table to store previously evaluated board states and avoid redundant calculations.

i. If so, how is it generated?

The transposition table is implemented as a hash table where the key is a hash of the board state, and the value is the evaluation of that state.

self.transposition_table[board_key] = (depth, max_eval)

ii. What do you store in it?

The transposition table stores the board state, its evaluation, and the depth at which it was evaluated.

iii. How do you use it in the search?

Before evaluating a board state, the algorithm checks if it exists in the transposition table. If it does, the stored evaluation is used instead of recalculating it.

d. What are the minimum, average, and maximum search depths observed by the algorithm?

The minimum depth is 1, and the maximum depth is 5, depending on the time limit. The algorithm uses iterative deepening, starting with depth 1 and increasing until the time limit is reached.

e. What is the branching factor of the game?

The branching factor is approximately 4, as each pawn has up to 4 possible moves (forward, diagonal capture, and en passant).

f. What is the effective branching factor of the agent?

The effective branching factor is around 2.5 due to pruning and move ordering.

Because Pruning reduces the number of nodes that need to be evaluated, lowering the effective branching factor.

g. Did you use forward and/or backward pruning?

Yes, the algorithm uses alpha-beta pruning, which is a form of backward pruning.

i. If so, what is the effect of pruning on the performance of the search algorithm?

Pruning significantly reduces the number of nodes that need to be evaluated, allowing the algorithm to search deeper within the time limit.By cutting off branches that are unlikely to be beneficial, the algorithm can explore more promising paths.

h. Did you use search extensions?

Yes, the algorithm uses quiescence search to extend the search in positions where captures are possible, avoiding the horizon effect.

i. If so, under what conditions does the algorithm extend the search?
The search is extended in positions where captures are possible, ensuring that the evaluation is not based on a volatile position.

ii. What is the criterion for stopping the extensions?
The quiescence search stops when no more captures are possible or when a maximum depth is reached.

4.Questions about Learning and Optimization

a. Did you use learning and/or optimization algorithms to improve your agent?

No, the agent does not use machine learning or optimization algorithms.

b. What was the method for testing the algorithm's performance during development?

The algorithm was tested by playing it against itself and against other agents and the one Shay gave to us , and its win rate and average depth of search were measured.

5.General Questions

a. How did the development process proceed?

The development process began with implementing the fundamental game rules, followed by designing the search algorithm, evaluation function, and optimization techniques such as alpha-beta pruning.

The project was developed incrementally, with each component being tested and refined before integrating the next. This iterative approach allowed for continuous improvements and debugging, ensuring a well-functioning AI agent.

b. What general principles about the Two Flags game did you deduce from the project experience?

Through this project, we discovered that the Two Flags game serves as an excellent introduction to AI-driven game development. It provided a structured environment to explore key AI principles such as strategic decision-making, heuristic evaluation, and algorithmic optimization.

By iteratively improving the agent, we observed firsthand how modifications to the algorithm and evaluation function directly impacted gameplay performance, reinforcing the importance of fine-tuning AI heuristics.

c. What conclusions did you draw from the project?

A well-optimized evaluation function and an efficient search algorithm are essential for developing a strong AI agent. The project highlighted the significance of balancing computational efficiency with strategic depth to create an AI that makes intelligent and adaptable decisions.

Additionally, we learned that AI performance is highly dependent on heuristic design, requiring careful analysis and refinement to improve gameplay strength.

d. What improvements do you see for the future?

Future improvements could include integrating machine learning techniques to refine the evaluation function and enhance the search algorithm's efficiency. By leveraging machine learning, the agent could adapt dynamically to different opponents and gameplay scenarios, making its strategy more robust and flexible.

Additionally, further optimization of the search algorithm could improve the agent’s decision-making speed, enabling better performance under time constraints while maintaining strategic depth.

***Running the project:**

We have attached a video demonstrating a game where our agent achieves a victory. The video serves as a practical example of how the agent performs in a real game scenario. Additionally, the video includes step-by-step instructions on how to run the project and the commands to interact with the server.

Commands to the Server:

- The video explains the commands that need to be sent to the server to start the game, such as:
- Time: To set the time limit for the game. Enter an integer (30)
- Setup: To initialize the board with specific pawn positions.

  Setup Wa2 Wb2 Wc2 Wd2 We2 Wf2 Wg2 Wh2 Ba7 Bb7 Bc7 Bd7 Be7 Bf7 Bg7 Bh7

  Or any setup you would like.
- Choose game mode 1 for Human vs Human, 2 for Human vs black agent, 3 for human vs white agent, 4 for white agent vs black agent

- Dictionary (weigh